Software Description Document

STAR: Spatial Topography Accessibility Robot

A Distributed Software Platform for Automated
ADA Title III Compliance Auditing

Team Locked Inc

Cesar Magana	Software, Project Manager
Brighton Sikarskie	Software, Embedded Development, Schematic Design
Jeremie Hockey	PCB Design and Layout
Michael Norton	Mechanical Design
Shawn Ciaciura	Schematic Review
Jared Bartz	Schematic Review

Course

ESET 420 – Senior Design II – Dr. Lee Hudson

Institution

Texas A&M University
Department of Electronic Systems Engineering Technology (ESET)
ESET Senior Capstone, Spring 2026

Advisors and Sponsors

Dr. Logan Porter – Sponsor and Technical Advisor

April 28, 2026

Version 1.0 – Capstone Submission

Contents

List of Figures

List of Tables

1 Executive Summary

The Spatial Topography Accessibility Robot (STAR) is an autonomous indoor robotic platform whose mission is to automate compliance auditing of facilities against the Americans with Disabilities Act (ADA) Title III. Title III enforcement remains active at scale: Seyfarth Shaw's ADA Title III tracker, sourced from PACER, reports federal lawsuit filings rose from 10,163 in 2018 to a peak of 11,452 in 2021, declined to 8,227 in 2023, and rebounded to 8,800 in 2024 – a sustained baseline of roughly nine to eleven thousand new federal suits per year over the seven-year window. Manual audits by certified inspectors remain expensive and slow, creating a structural gap between the volume of physical spaces under enforcement risk and the rate at which they can be measured and remediated. STAR exists to close that gap by replacing discretionary manual measurement with deterministic, sensor-grounded, repeatable software measurements.

This document describes the software that realizes that mission. The STAR software stack is a distributed system spanning five deployment targets: a React-based operator dashboard, a Go gateway service on a Raspberry Pi 5, a ROS2 Jazzy autonomy layer, a Renesas RX72N real-time motor-control firmware written in C23, and a Python ROS2 compliance-audit engine that applies ADA 405.2 (and related) analytical checks to fused LiDAR, IMU, and odometry data. A five-layer communication stack binds these components together: SPI at 10 MHz at the physical layer, a framed transport with CRC-32 integrity, a rate-1/2 constraint-length-7 convolutional forward error-correction (FEC) layer with soft-decision Viterbi decoding, a Chase Combining hybrid automatic repeat request (HARQ) retransmission layer, and a gRPC service layer.

Transport status note. The full SPI-plus-FEC-plus-HARQ-plus-gRPC stack described above is the *designed* transport and is the status [ARCHITECTED] target for production. The transport *deployed* for the capstone defense, used by the Pi5 cockpit and the demonstrated SLAM/Nav2 stack, is a simpler ASCII line protocol over USB-CDC at 115200 baud via `star_simple_bridge` (see Section on the ROS2 autonomy workspace). The two transports share the same Protocol Buffers contract at the message level. Where this SDD describes SPI / FEC / HARQ / gRPC end-to-end, those layers should be read as [ARCHITECTED] unless the surrounding text calls out a deployed implementation explicitly. The SSUM (operator-facing companion document) describes only the deployed USB-CDC bridge, which is why the two documents differ on the RX72N-side transport.

The software decomposes into eight distinct subsystems: a language-neutral Protocol Buffers contract (`star-proto`), the RX72N motor-control firmware, the Go gateway service, the ROS2 autonomy workspace, the React operator UI, offline MATLAB scripts for motor system identification and PID gain design, the Pi5 cockpit API plus provisioning scripts, and the ADA compliance-audit engine. The STM32 peripheral and BeagleBone Blue bring-up targets are intentionally outside the scope of this SDD.

STAR's core engineering discipline is safety-critical: zero dynamic memory allocation on the RX72N, NASA Power of 10 coding rules, strict 7-bit ASCII source enforcement, static analysis via clang-tidy and CodeRabbit, and a pre-commit hook pipeline that rejects any source file containing non-ASCII characters. Real-time control runs at a 250 Hz motor servo rate with a velocity-form discrete PID derived from a MATLAB-identified first-order plant $G(s) = 3.665/(0.075s + 1)$.

1.1 Team Contributions

Responsibility for the eight STAR software subsystems is distributed across the team as summarized in Table 1. Each subsystem has a primary owner who is accountable for its architecture, imple-

mentation, and test coverage, and a secondary reviewer who performs code review and integration verification.

Table 1: Team subsystem ownership and accountability matrix.

Subsystem	**Primary Owner**	**Secondary / Reviewer**
star-proto (API contract)	Cesar Magana	Brighton Sikarskie
star-rx72n-firmware	Brighton Sikarskie	Cesar Magana
star-gateway (Go)	Cesar Magana	Brighton Sikarskie
star-ros2 (autonomy)	Cesar Magana	Brighton Sikarskie
star-ui (React)	Cesar Magana	Brighton Sikarskie
matlab/ (sys-ID and PID)	Cesar Magana	Brighton Sikarskie
infra/ + scripts/ (cockpit)	Brighton Sikarskie	Cesar Magana
compliance-engine (ADA)	Brighton Sikarskie	Cesar Magana

2 Introduction

2.1 Purpose

This Software Description Document (SDD) specifies the architecture, design, interfaces, and implementation of the software that runs on the STAR robotic platform. It is intended to serve three audiences: (1) capstone course reviewers performing end-of-semester evaluation, (2) follow-on student teams inheriting the codebase, and (3) outside integrators who may build on STAR's open-source software in future derivative work.

2.2 Scope

The document covers eight STAR software subsystems: the Protocol Buffers schema repository, the RX72N real-time firmware, the Go gateway service, the ROS2 Jazzy autonomy workspace, the React operator dashboard, the MATLAB motor-modeling and PID-design environment, the Raspberry Pi 5 cockpit API and provisioning infrastructure, and the Python compliance-audit ROS2 engine.

The following are explicitly *out of scope*:

- STM32 peripheral firmware (covered in a separate hardware deliverable).
- BeagleBone Blue bring-up and secondary compute work (retired target; retained only as a historical reference).
- Mechanical CAD, PCB schematics, and power electronics design.
- Field-trial results data sets and published experimental data.

2.3 Definitions and Acronyms

Table 2 lists abbreviations used throughout this document. Longer glossary entries appear in Appendix 14.

Table 2: Common acronyms used in this document.

Acronym	Expansion
ADA	Americans with Disabilities Act
CRC	Cyclic Redundancy Check
DDS	Data Distribution Service (ROS2 transport)
EKF	Extended Kalman Filter
FEC	Forward Error Correction
HAL	Hardware Abstraction Layer
HARQ	Hybrid Automatic Repeat Request
IMU	Inertial Measurement Unit
PID	Proportional–Integral–Derivative controller
PPR	Pulses Per Revolution (quadrature encoder)
RANSAC	Random Sample Consensus
RPM	Revolutions Per Minute
RTOS	Real-Time Operating System
SLAM	Simultaneous Localization and Mapping
SPI	Serial Peripheral Interface
TF	Transform frame tree (ROS2 REP-105)

2.4 Document Conventions

Source code, filenames, and shell commands appear in `monospace`. Protocol Buffers types appear as `PascalCase` identifiers. ROS2 topics and services use a leading slash: `/compliance/report`. Numeric units follow the MKS system with explicit suffixes; rates are expressed in hertz. All source files in the STAR repository are strictly 7-bit ASCII; this document follows the same constraint.

2.5 Intended Audience

The primary audience is the capstone review committee. Follow-on student teams should treat Sections 5 and 7 as their onboarding contract. Open-source integrators interested in reusing the compliance-engine independently should focus on Section 5 subsystem “Compliance Engine” and Section 6.

3 System Overview

3.1 Problem Statement

The 2010 ADA Standards for Accessible Design, Section 405.2 (incorporated under DOJ Title III regulation at 28 CFR Part 36), require that newly constructed or altered ramp runs have a running slope no steeper than 1:12. In practice, inspectors audit this and related geometric constraints by hand, using inclinometers, tape measures, and photography. Manual auditing is slow, subjective, and error-prone. Federal Title III lawsuit filings have remained at a sustained high baseline – between roughly 8,200 and 11,500 per year from 2018 through 2024 (Seyfarth Shaw ADA Title III tracker, 2025) – yet the rate of formal audit completions has not kept pace. STAR addresses this gap by combining an autonomous mobile platform with a deterministic software pipeline that measures ramp slope, path width, trip-hazard height, and related ADA geometry from sensor data with known tolerances.

3.2 System Context

STAR is a four-motor differential-plus-caster mobile base carrying a Raspberry Pi 5 host computer, a Renesas RX72N motor-control microcontroller, an RPLiDAR C1 2D lidar, a BNO055 IMU, and a BMP280 barometric-pressure sensor. At runtime an operator opens the STAR web UI in a browser, which connects over HTTPS and WebSocket to the gateway service on the Pi5. The gateway exposes gRPC-web endpoints that invoke ROS2 actions, and in parallel it maintains a Protocol Buffers SPI channel to the RX72N firmware for motor commands and telemetry. Figure 1 shows the end-to-end data path.

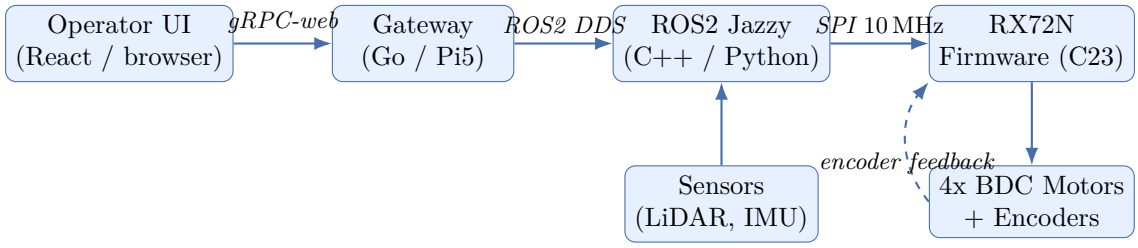

Figure 1: STAR system context. Commands flow left to right; telemetry flows right to left.

3.3 Deployment Topology

The host computer is a Raspberry Pi 5 running Ubuntu 24.04 with ROS2 Jazzy installed from the OSRF apt repository. The gateway binary runs as a systemd unit on the Pi5, bound to loopback for gRPC and to a TLS-terminated Caddy reverse proxy for WebSocket and HTTP. The RX72N is mounted on the custom motherboard PCB and communicates with the Pi5 via a dedicated SPI bus. Sensors attach to the Pi5 directly: the RPLiDAR C1 over USB, the IMU over I2C through the RX72N's bus manager.

4 Software Architecture

4.1 Architectural Style

STAR is organized as a *layered service-oriented hybrid*. The layered aspect applies to the communication stack: the operator UI sits above the gateway, which sits above ROS2, which sits above the RX72N firmware. The service-oriented aspect applies horizontally within each layer: the gateway is composed of five independent gRPC services (motor control, telemetry, configuration, gateway control, firmware update), and the ROS2 autonomy layer is composed of eight packages that communicate only through DDS topics and actions.

Figure 2 shows the eight software subsystems and their principal dependencies.

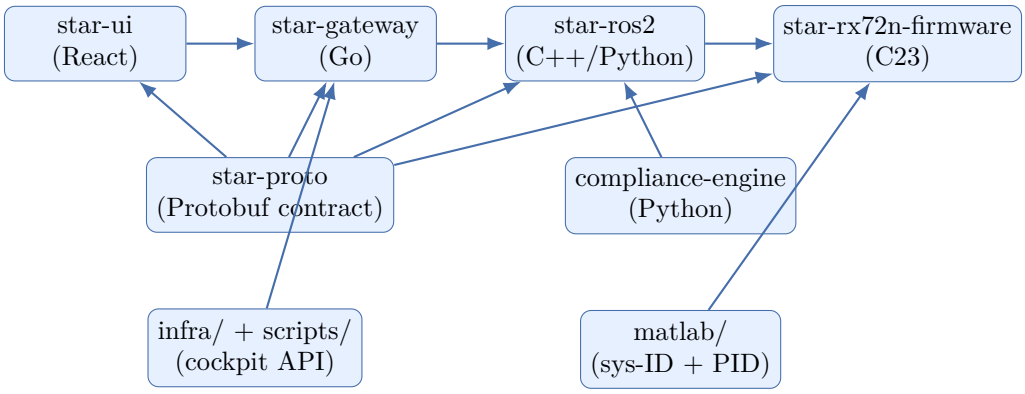

Figure 2: Subsystem component diagram. Arrows denote "depends on" relationships at build time or runtime.

4.2 Subsystem Responsibilities

The eight subsystems partition the codebase along clear responsibility boundaries:

- **star-proto** owns the language-neutral API contract and emits Go, TypeScript, C (nanopb), and C++ bindings.
- **star-rx72n-firmware** owns hard-real-time motor control, encoder decoding, IMU reading, and SPI peripheral framing.
- **star-gateway** owns the five-layer protocol stack and bridges the UI to both ROS2 and the RX72N.
- **star-ros2** owns SLAM, sensor fusion, Nav2 planning, lifecycle-managed safety monitoring, and action servers.
- **star-ui** owns all user-facing operator workflows: teleoperation, autonomy mode switching, telemetry plotting, compliance-report viewing.
- **matlab** owns offline motor system identification and PID gain design; it exports gains to the firmware via a generated header.
- **infra/ + scripts/** owns the Pi5 cockpit REST API, Caddy reverse-proxy configuration, and field-provisioning shell scripts.
- **compliance-engine** owns the ADA 405.2 and related geometric audit checks running as ROS2 Python nodes.

4.3 Cross-Cutting Concerns

Five cross-cutting concerns are enforced across every subsystem: **error handling** is return-value-based in C and Go (no exceptions), exception-based in C++ ROS2 nodes, and Promise-based in TypeScript; **logging** uses the native facility of each runtime (`printf`-like `rx_log_*` on RX72N, structured logs in Go, `RCLCPP_*` in C++, a ring-buffer alert bus in React); **unit convention** is MKS throughout with explicit suffixes (`_mps`, `_rad`, `_ma`, `_celsius`); **character encoding** is 7-bit ASCII only, enforced at commit time; and **memory allocation** on the RX72N is statically bounded, with zero `malloc` or `free` calls after the boot sequence.

5 Component and Module Descriptions

This section describes each subsystem in turn. For each, we cover purpose, internal modules, primary inputs and outputs, dependencies, and key algorithmic logic.

5.1 star-proto (API Contract)

Purpose: Provide a single source of truth for all STAR message and service definitions, and emit language bindings for the four downstream runtimes.

Inventory: Ten Protocol Buffers files under `star-proto/proto/star/v1/`: `common.proto`, `configuration.proto` `controller.proto`, `diagnostics.proto`, `firmware_update.proto`, `gateway_service.proto`, `motor_control.pr` `telemetry.proto`, `ui.proto`, and `wire.proto`.

Code generation: Managed by the Buf CLI (version 1.28.1) via `buf.gen.yaml`. Go bindings use `buf.build/protocolbuffers/go` and `buf.build/grpc/go`; TypeScript uses `timostamm-protobuf-ts` (2.11.1); C bindings are generated by `nanopb_generator`; C++ bindings use the standard `protoc-gen-cpp`.

nanopb field sizing: RX72N has no dynamic allocation, so every variable-length field must declare a compile-time maximum via an adjacent `.options` file. Example: `star.v1.RequestHeader.request_id max_size:64`.

Style: Boston Dynamics Proto3 conventions: `PascalCase` messages, `snake_case` fields, `SCREAMING_SNAKE` enums with a zero value ending in `_UNKNOWN`, 100-character line limit, MKS unit suffixes, and a `RequestHeader`/`ResponseHeader` pair on every RPC message.

5.2 star-rx72n-firmware (Real-Time Motor Control)

Purpose: Provide deterministic closed-loop control of four brushed DC gearmotors while serving as an SPI peripheral to the Pi5 gateway.

Platform: Renesas RX72N, 32-bit RX CPU core at 240 MHz (ICLK), 4 MB Flash, 512 kB SRAM, GNURX toolchain, CMake-driven build.

RTOS: Azure RTOS ThreadX with seven statically allocated tasks, shown in Table 3.

Table 3: ThreadX tasks on RX72N firmware.

Task	Priority	Stack	Role
CommTask	P5	2 kB	SPI framer, nanopb codec
MotorControlTask	P12	2 kB	250 Hz PID servo
WatchdogMonitorTask	P20	1 kB	RTOS + WDT health
ObstacleDetectTask	P22	1.5 kB	Sonar + proximity fusion
TempSensorTask	P24	1 kB	BMP280 + thermistor
LEDStatusTask	P25	0.5 kB	Status LED state machine
TelemetryTask	P26	1 kB	Diagnostics aggregation

HAL modules: CMT (compare-match timer), GPTW (four channels of 20 kHz motor PWM with 1 µs hardware dead-time), TPU (quadrature encoder decoding at 341 PPR), RSPI (SPI peripheral mode), SCI (debug UART through the CY7C65213 USB-UART bridge), RIIC (BNO055 IMU and BMP280 barometer), hardware CRC-32 (polynomial `0xEDB88320`, approximately 32 µs for 100 bytes), DMACA, MPC (multi-function pin controller), WDT and IWDT, and the POEG port output-enable gate used as an H-bridge fail-safe kill switch.

Motor driver: Each of four channels drives a DRV8263H dual H-bridge in IN/IN PWM mode. The startup sequence is DRVOFF high, `nSLEEP` high, wait `tWAKE`, then DRVOFF low.

Library sizes (LOC): `rx_hal` 57.7k, `threadx` 43.5k, `rx_core` 26.7k, `rx_bus` 14.9k, `rx_nanopb` 7.2k, `rx_spi_comm` 4.2k, `rx_comm_manager` 3.6k, `rx_encoder` 3.4k, `rx_motor` 2.6k, `rx_pid` 2.3k; roughly 71k LOC total across the firmware libraries.

Error convention: `rx_err_t` is a signed 32-bit integer. Code ranges: `0x000` success, `0x100-0x1FF` generic, `0x200-0x2FF` hardware, `0x300-0x3FF` RTOS, `0x400-0x4FF` communication, `0x500-0x5FF` validation.

Boot budget at 240 MHz: Approximately 50 ms from reset to `tx_kernel_enter()` (reset-flag check, clock init, hardware init, USB enum, kernel start).

5.3 star-gateway (Go Protocol Bridge)

Purpose: Bridge between the operator UI and both the RX72N SPI peripheral and the ROS2 autonomy layer. Runs as a systemd service on the Pi5.

Stack: Go 1.26.2, `google.golang.org/grpc` v1.80.0, `gorilla/websocket` for WebSocket transport to the UI, and `periph.io` v3 for Linux SPI device access.

Five-layer protocol stack (from application down to hardware): gRPC service handlers, Chase Combining HARQ retransmission, rate-1/2 K=7 convolutional FEC, framing (sync + length + sequence + CRC-32), SPI 10 MHz (CPOL=0, CPHA=0) transport.

gRPC services: `MotorControlService`, `TelemetryService`, `ConfigurationService`, `GatewayService`, `FirmwareUpdateService`.

Environment flags: `WS_STRICT_ORIGIN` toggles WebSocket origin checking, `STAR_CDC_DEVICE` selects the USB-CDC fallback device for firmware flashing, `STAR_SIMULATION_MODE` routes SPI traffic through an in-process virtual RX72N, and `TRANSPORT_MODE` selects SPI, CDC, or simulated transport at startup.

5.4 star-ros2 (Autonomy)

Purpose: Perception, localization, mapping, path planning, and safety monitoring.

Distribution: ROS2 Jazzy on Ubuntu 24.04, C++20 with `rclcpp`, plus Python nodes where appropriate.

Packages (10 total in `star-ros2/src/`): `star_bringup` (launch orchestration), `star_simple_bridge` ([IMPLEMENTED] – the deployed USB-CDC ASCII bridge to the RX72N at 115200 baud; serves OccupancyGrid PNG and Prometheus metrics on `:9101`), `star_spi_bridge` ([ARCHITECTED] – the production SPI bridge target; not used at defense), `star_gateway_bridge`, `star_safety_monitor` (lifecycle-managed emergency stop), `star_perception` (point-cloud preprocessing), `star_simulation` (SITL harness), `star_supervisor` (top-level state coordinator), `star_compliance_msgs` (custom message definitions for the ADA compliance engine), and `sllidar_ros2` (vendor driver for the RPLiDAR C1).

Third-party stack: `slam_toolbox` (async SLAM with loop closure), `robot_localization` (EKF fusing `/odom/unfiltered` with `/imu/data`), Nav2 for occupancy-grid path planning, and `explore_lite` for frontier exploration.

TF tree: `map -> odom -> base_link -> {laser_frame, imu_link}` following REP-105.

5.5 star-ui (Operator Dashboard)

Purpose: Provide a single-page browser application for teleoperation, autonomy mode selection, live telemetry plotting, and compliance-report viewing.

Stack: React 19.2.0, TypeScript 5.9, Vite 7.2.4 (build), Zustand 5.0.3 (state), uPlot 1.6.31 (GPU-accelerated time-series plots), `react-grid-layout` (dashboard panel placement), and gRPC-web (telemetry streams and command unary RPCs).

Panel inventory: Seventeen panels, including live map view, command velocity joystick, four motor plots, IMU orientation display, LiDAR scan overlay, alert log ring buffer, autonomy mode switch, emergency-stop panel, SLAM control, firmware-update panel, and the compliance-report viewer.

Reconnection: WebSocket transport implements exponential backoff reconnection with jitter; the frontend degrades gracefully to "stale" indicators after the first reconnect attempt exceeds one second.

5.6 matlab (Motor System Identification and PID Design)

Purpose: Offline identification of the motor-plant transfer function from step-response data, and deterministic computation of PID gains that satisfy the firmware's real-time constraints.

Identified plant: The open-loop transfer function from input voltage to angular velocity is

$$G(s) = \frac{3.665}{0.075s + 1}$$

a first-order model with DC gain $3.665\,\mathrm{rad/(s\,V)}$ and mechanical time constant $\tau = 0.075\,\mathrm{s}$ (75 ms). The identification is performed in the MATLAB System Identification Toolbox on a step-response data set captured at the 250 Hz firmware sample rate.

Validation plots: The MATLAB workflow produces three validation figures. The first overlays measured versus modeled step response and demonstrates that the closed-loop response settles within two seconds with an overshoot under ten percent for the tuned gains. The second is a Bode plot showing a flat low-frequency magnitude asymptote at the identified DC gain and a single real-pole rolloff at approximately 13.3 rad/s (equal to $1/\tau$ with $\tau = 0.075\,\mathrm{s}$). The third is a closed-loop disturbance-rejection plot, confirming that the chosen gains reject a simulated load step without oscillatory return. The plot set constitutes the go/no-go checkpoint before new gains are exported to the firmware.

Why offline: Three reasons. First, safety: autotuning on hardware with untrusted gains risks winding the motor against mechanical stops. Second, reproducibility: the MATLAB script deterministically produces the same gains from the same data, which allows regression review across semesters. Third, export determinism: gains are written via `pid_config_output.txt` into a header consumed by the firmware build, so a rebuild is the only coupling between controller design and deployed code.

5.7 infra/ and scripts/ (Pi5 Cockpit)

Purpose: Minimal facility for operators to observe and control the Pi5 host itself, independent of ROS2.

Stack: Python 3 Flask service at `:9102`, which imports `rclpy` for read-only ROS2 introspection. Caddy is fronted on port 443 with a local certificate authority for development. Systemd unit files

run both Caddy and the cockpit API with sandbox directives (`NoNewPrivileges`, `PrivateTmp`, `ProtectSystem`).

Endpoints: `/api/state` (aggregate health status), `/api/autonomy/toggle`, `/api/estop/toggle`, and the `/api/slam/*` family for start, stop, and map save.

Scripts: `scripts/` carries provisioning, build, flash, and maintenance automation in Bash and Python, all under pre-commit-enforced 7-bit ASCII.

5.8 Compliance Engine (ADA Audit Core)

Status-label legend (applies throughout this document; every `[IMPLEMENTED]` / `[STRETCH]` / `[ARCHITECTED]` tag elsewhere in the SDD links back here): *[IMPLEMENTED]* means the feature is complete and has passing tests; *[STRETCH]* means a working stub with scaffolding in place but without the full algorithm; *[ARCHITECTED]* means specified and designed but not yet coded.

Purpose: Apply ADA 405.2 and neighboring geometric checks to fused point-cloud, odometry, and IMU data and emit a structured compliance report.

Stack: Python 3, ROS2 `rclpy` nodes under `final_capstone/compliance-engine/`, packaged with `setuptools` and `ament_python`. Numerical work uses Open3D and NumPy.

Checks: all eleven nodes below ship as files under `final_capstone/compliance-engine/star_compliance/nodes` the status label tracks the algorithmic completeness of each.

- **Ramp slope** (`ramp_slope_node.py`) – [IMPLEMENTED]. RANSAC plane fit to LiDAR points in a ramp region of interest, cross-validated against RX72N IMU pitch.
- **Trip hazard** (`trip_hazard_node.py`) – [STRETCH stub]. Detects vertical discontinuities above the ADA threshold (one quarter inch).
- **Path width** (`path_width_node.py`) – [STRETCH stub]. Occupancy-grid cross-section width along the planned path.
- **Ramp width** (`ramp_width_node.py`), **ramp landing** (`ramp_landing_node.py`), **door clear width** (`door_clear_width_node.py`), **door threshold** (`door_threshold_node.py`), **path blockage** (`path_blockage_node.py`), **protruding objects** (`protruding_objects_node.py`), **dynamic obstacle** (`dynamic_obstacle_node.py`), and the supervising **compliance monitor** (`compliance_monitor_node.py`) – [ARCHITECTED stubs]. Node skeletons, message schemas, and traceability-matrix rows are committed to the repo; the internal algorithm bodies are placeholders pending post-capstone work.

Core engines: `engines/plane_segmentation.py` implements the RANSAC plane extractor used by the ramp checks, and `engines/imu_cross_validate.py` reconciles the measured plane slope against IMU pitch to flag suspected sensor drift.

Reporting: The `report/` package generates a PDF audit report; an example is stored at `star_audit_report_samp` Structured per-check messages are published on `/compliance/report` and the per-check topics enumerated in Section 7.

Tests: `compliance-engine/tests/` contains `pytest` unit tests for the engines and a small set of golden-output integration tests. The traceability matrix at `final_capstone/extras/traceability.md`

maps every ADA check row to its sensor, algorithm, node, output topic, and status label, and is reproduced in Appendix 14.

6 Data Design

6.1 Protocol Buffers Schema Inventory

The ten proto files under `star-proto/proto/star/v1/` are summarized in Table 4.

Table 4: Protocol Buffers file inventory.

File	Role
`common.proto`	Shared envelopes (RequestHeader, ResponseHeader, Status)
`configuration.proto`	PID and calibration payloads
`controller.proto`	Operator command messages
`diagnostics.proto`	Health and telemetry diagnostics
`firmware_update.proto`	Firmware image + CRC metadata
`gateway_service.proto`	Gateway control RPCs
`motor_control.proto`	Per-motor commands + feedback
`telemetry.proto`	Streaming telemetry schema
`ui.proto`	UI-specific envelopes
`wire.proto`	Transport-layer framing types

6.2 Wire Frame Layout

Table 5 describes the physical frame on the SPI wire between the Pi5 and the RX72N.

Table 5: SPI wire-frame layout (little-endian on the wire).

Field	Size (bytes)	Description
Sync	2	Fixed pattern `0xA55A`
Length	2	Payload byte count, unsigned
Seq	2	Monotonic sequence number, wraps
Payload	N	Nanopb-encoded protobuf message
CRC-32	4	Polynomial `0xEDB88320` over Length..Payload

6.3 UI State Shape

The React operator dashboard uses a single Zustand store split into four top-level slices: `connection` (transport health, reconnect state), `telemetry` (motor, IMU, battery), `autonomy` (Nav2 state, SLAM state), and `alerts` (a 200-entry capped ring buffer of UI and transport alerts). Each slice is serializable for debug export.

6.4 ROS2 Topic and TF Graph

The canonical topic set is listed in Section 7 Table 6. The transform tree follows REP-105 with `map -> odom -> base_link`, and `base_link` parents the two sensor frames `laser_frame` and `imu_link`.

6.5 Compliance Report Schema

Each compliance run emits a `ComplianceReport` message containing a facility identifier, a run timestamp, and a list of `CheckResult` entries. A `CheckResult` carries the check identifier (for example, `ADA_405_2_RAMP_SLOPE`), the measured value, the ADA threshold, a pass / fail / insufficient-data verdict, and a sensor-provenance record.

6.6 Protocol-Stack Data Flow

Figure 3 traces the transmit path from an operator action in the UI down to the electrical SPI signals on the RX72N pins. The receive path mirrors this diagram in reverse.

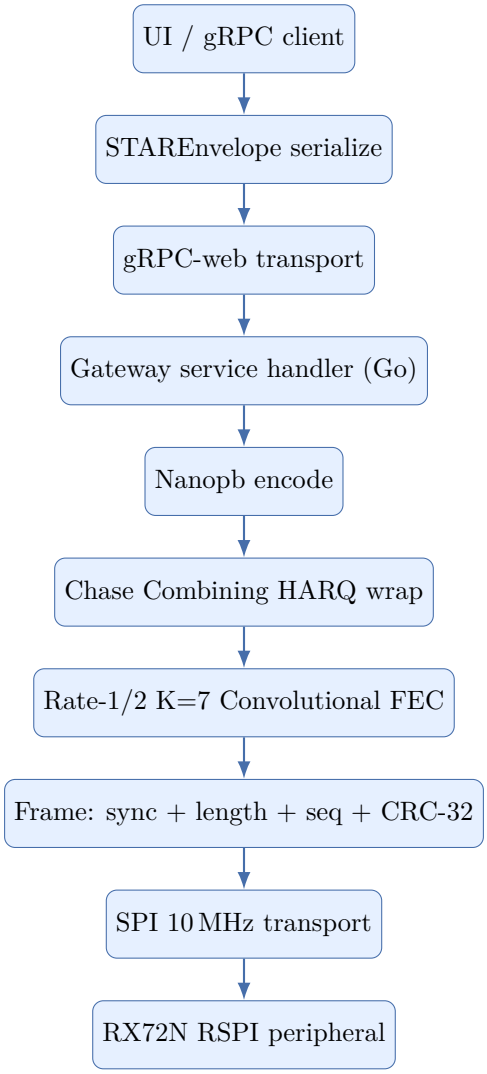

Figure 3: Protocol-stack data flow (transmit path, top-to-bottom). The receive path mirrors this diagram in reverse. HARQ and FEC sit below the gRPC transport on the transmit side; they are not wrapped around gRPC.

7 Interface Design

7.1 SPI Physical Layer

Shared bus between the Pi5 controller and the RX72N peripheral. Clock rate 10 MHz, CPOL = 0, CPHA = 0, MSB-first, 8-bit word. Chip select is driven by a single GPIO on the Pi5. The Pi5 side uses the `periph.io` SPI driver, and the RX72N side uses RSPI in peripheral (slave) mode with DMA-backed ping-pong buffers.

7.2 gRPC Services

Five services are defined in the proto inventory. Every request carries a `RequestHeader` and every response carries a `ResponseHeader`.

- `MotorControlService` – `SetVelocity`, `SetPosition`, `EmergencyStop`, `GetMotorStatus`.
- `TelemetryService` – `StreamTelemetry` (server streaming), `SnapshotTelemetry` (unary).
- `ConfigurationService` – `GetConfig`, `SetConfig`, `CommitConfig`.
- `GatewayService` – `Ping`, `GetStatus`, `Restart`, `SwitchTransport`.
- `FirmwareUpdateService` – `StartUpdate`, `StreamImage`, `FinalizeUpdate`.

7.3 WebSocket Binary Envelope

The UI connects to the gateway using a WebSocket upgrade on top of Caddy TLS. All frames are binary (opcode 2) and carry a single `STAREnvelope` message. The envelope discriminates between telemetry frames, alert frames, and command frames by a tagged oneof.

7.4 ROS2 Topic Inventory

Table 6 lists the canonical ROS2 topics that cross package boundaries, including all compliance-engine topics.

Table 6: ROS2 canonical topic inventory.

Topic	Type	Role
`/cmd_vel`	`geometry_msgs/Twist`	Velocity command
`/odom/unfiltered`	`nav_msgs/Odometry`	Wheel odometry pre-fusion
`/odom/filtered`	`nav_msgs/Odometry`	EKF output
`/imu/data`	`sensor_msgs/Imu`	BNO055 fused IMU
`/scan`	`sensor_msgs/LaserScan`	RPLiDAR C1 scan
`/map`	`nav_msgs/OccupancyGrid`	SLAM output
`/diagnostics`	`diagnostic_msgs/...`	Health aggregator
`/compliance/report`	`star_msgs/Report`	Aggregate audit result
`/compliance/ramp_slope`	`star_msgs/Check`	Per-check result
`/compliance/trip_hazard`	`star_msgs/Check`	Per-check result
`/compliance/path_width`	`star_msgs/Check`	Per-check result

7.5 Pi5 Cockpit REST API

Four primary REST endpoints are exposed by the Flask cockpit on `:9102` behind Caddy TLS: `/api/state` (GET, aggregate health), `/api/autonomy/toggle` (POST, toggle autonomy mode),

`/api/estop/toggle` (POST, engage or clear emergency stop), and `/api/slam/{start,stop,save}` (POST).

7.6 UI Panel Inventory

The seventeen React panels include: live 2D map, LiDAR overlay, joystick teleoperation, four per-motor live plots (one per wheel), IMU orientation, battery, temperature, alert log, autonomy mode switch, emergency stop, SLAM control, firmware update, compliance summary, compliance per-check, system health, and a diagnostic raw log panel for debugging.

8 Technology Stack and Justifications

Table 7 enumerates the technology choices made for the STAR software project, together with a short justification. Version numbers are pinned by the top-level lockfiles and package manifests.

Table 7: Technology stack and selection rationale.

Subsystem	**Choice**	**Rationale**
Gateway	Go 1.26.2	First-class gRPC, small binaries, cgo-free SPI via periph.io
Firmware	C23, GNURX, ThreadX	Vendor toolchain support, C23 typed enums, small RTOS footprint
ROS2 distribution	Jazzy (Ubuntu 24.04)	Long-term support release matched to rclcpp C++20
UI framework	React 19.2 + Vite 7	Concurrent rendering, fast HMR, mature component ecosystem
UI state	Zustand 5.0.3	Minimal API, no boilerplate, selector memoization
UI plots	uPlot 1.6.31	GPU-friendly time-series plots, tiny bundle
Proto build	Buf 1.28.1	Reproducible lint and breaking-change detection
TypeScript bindings	protobuf-ts 2.11.1	First-class gRPC-web, clean TS ergonomics
C bindings	nanopb	Zero dynamic allocation, proto3 subset
Compliance nodes	rclpy (Python 3)	NumPy/Open3D access, rapid iteration on algorithms
MATLAB	Control + System ID	Canonical offline sys-ID and PID design workflow
Reverse proxy	Caddy	Automatic TLS with local CA, small config surface
Cockpit API	Flask + rclpy	Single-file REST, trivial systemd integration

Specific reasons for non-obvious choices:

Why ThreadX over FreeRTOS? Renesas distributes vetted ThreadX ports for the RX72N with guaranteed memory footprints. The static-allocation discipline aligns naturally with NASA Power of 10 Rule 3.

Why Buf over `protoc` directly? Buf enforces style rules, detects breaking changes on every push, and produces reproducible generated artifacts across CI runners.

Why protobuf-ts instead of ts-proto? Better gRPC-web support, cleaner oneof ergonomics, and an active maintainer release cadence.

Why rclpy for compliance, not C++? The compliance engine is algorithmically dominated by point-cloud work (Open3D, NumPy) where Python has mature bindings, and real-time constraints do not apply: compliance runs at mission cadence, not loop cadence.

9 Key Algorithms and Logic

9.1 Cascaded PID (Position Outer, Velocity Inner)

Each motor is controlled by a two-stage cascade. The outer position loop produces a velocity setpoint; the inner velocity loop produces a voltage setpoint that becomes a PWM duty ratio. In continuous time each stage is a standard PID:

$$u(t) \;=\; K_p\, e(t) \;+\; K_i \int_0^t e(\tau)\, d\tau \;+\; K_d\, \frac{de(t)}{dt}.$$

In firmware we use the velocity form of the PID at sample period $T_s = 0.004\,\mathrm{s}$ (250 Hz):

$$u[k] = u[k-1] + K_p(e[k] - e[k-1]) + K_i T_s\, e[k] + \tfrac{K_d}{T_s}\big(e[k] - 2e[k-1] + e[k-2]\big).$$

The equivalent discrete transfer function is

$$C(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 - z^{-1}}.$$

Anti-windup uses integral clamping to the configured saturation bounds; the derivative term is low-pass-filtered to limit sensor noise amplification.

```
function pid_update(state, setpoint, measured, dt):
    e[k]   <- setpoint - measured
    du     <- Kp*(e[k] - state.e_km1)
            + Ki*dt*e[k]
            + (Kd/dt)*(e[k] - 2*state.e_km1 + state.e_km2)
    u_new  <- clamp(state.u_prev + du, u_min, u_max)
    state.e_km2 <- state.e_km1
    state.e_km1 <- e[k]
    state.u_prev <- u_new
    return u_new
```

Listing 1: Velocity-form PID update.

9.2 Quadrature Encoder Decoding

The TPU peripheral is configured as a signed up/down counter clocked by quadrature-decoded A/B edges. At each control tick the firmware reads the counter, computes a signed delta modulo 2^{16} to handle wraparound, and multiplies by the calibrated 1 count-to-radian conversion (341 PPR, 4x decoded = 1364 edges per revolution, and an optional gearbox ratio).

9.3 Convolutional FEC and Viterbi Decoding

The gateway applies a rate-1/2 convolutional encoder with constraint length $K = 7$ and generator polynomials $G_1 = 171_8$, $G_2 = 133_8$. The decoder runs soft-decision Viterbi with a traceback depth of 35 (5K rule of thumb). Soft bits from the SPI front-end are quantized to four bits per symbol before entering the branch-metric accumulator.

9.4 Chase Combining HARQ

On a CRC failure the receiver retains the soft-bit vector for the failed frame, and on retransmission the soft bits are added elementwise before Viterbi decoding. This provides approximately 3 dB of combining gain after a single retransmission and monotonically improves with further retransmissions up to the configured cap.

9.5 Sensor Fusion (robot_localization EKF)

`robot_localization` runs an extended Kalman filter at 50 Hz fusing wheel odometry (pose and twist) with IMU orientation and angular-rate channels. The output is published on `/odom/filtered` and consumed by SLAM and Nav2.

9.6 SLAM and Exploration

`slam_toolbox` in asynchronous mode builds the occupancy grid from the LiDAR scan, with pose-graph optimization performing loop closure when revisiting landmarks. `explore_lite` selects frontier cells (occupancy-grid boundaries between known-free and unknown cells) as the next navigation goal until no unexplored frontier remains.

9.7 WebSocket Reconnection

The UI wraps its WebSocket in an exponential-backoff reconnect loop: delay $d_n = \min(d_{max}, d_0 \cdot 2^n) + \text{jitter}$ with $d_0 = 250\,\text{ms}$, $d_{max} = 15\,\text{s}$, and jitter drawn uniformly from 0 ms to 250 ms.

9.8 RANSAC Plane Segmentation for ADA 405.2

The compliance engine extracts the dominant plane from a LiDAR point cloud using RANSAC: it samples three random points, fits a plane, scores the plane by the count of inliers within a configured tolerance, and iterates up to a bounded number of samples. The final plane's normal vector determines the slope relative to gravity (cross-checked against the IMU pitch). Pseudocode appears in Listing 2.

```
function ransac_plane(points, tol, n_iter):
    best_plane  <- null
    best_inliers <- 0
    for i in 1..n_iter:
        sample  <- random_choice(points, 3)
        plane   <- plane_from_three_points(sample)
        inliers <- count(p in points where dist(p, plane) < tol)
        if inliers > best_inliers:
            best_inliers <- inliers
            best_plane   <- plane
    return best_plane
```

Listing 2: RANSAC plane fit for ramp slope.

10 Error Handling and Logging

10.1 Error Propagation

The RX72N firmware uses `rx_err_t` (signed 32-bit) return codes exclusively; no C++ exceptions cross the RTOS boundary. Propagation uses the `RX_RETURN_ON_ERROR` macro, which inspects the return value, logs a tagged error, and early-returns if nonzero.

The Go gateway uses the standard `fmt.Errorf("context: %w", err)` wrap idiom and converts to gRPC status codes at service boundaries using `status.Code`.

The ROS2 C++ layer propagates exceptions within a node and converts them to lifecycle transitions or `/diagnostics` publications at node boundaries.

The UI surfaces errors through a ring-buffer alert log capped at 200 entries; overflow discards the

oldest entries. Each alert carries a severity (info, warning, error), a tag, a timestamp, and an optional structured payload.

10.2 Logging Conventions

- RX72N: `rx_log_error`, `rx_log_warn`, `rx_log_info`, `rx_log_debug`; level-gated at compile time via `LOG_LEVEL`.
- Go: structured logs via the standard `log/slog` package, with a JSON handler in production.
- ROS2: `RCLCPP_INFO`, `RCLCPP_WARN`, `RCLCPP_ERROR` with per-node named loggers.
- UI: the Zustand `alerts` slice plus a dev-only `console.debug` channel.

10.3 Watchdog Strategy

Both WDT and IWDT are enabled on the RX72N. The `WatchdogMonitorTask` feeds the WDT and verifies that every ThreadX task has checked in within a configured timeout. Missed check-ins escalate to a POEG-driven H-bridge shutdown, which stops the motors within microseconds.

11 Security Considerations

11.1 WebSocket Origin and Transport

The gateway defaults to `WS_STRICT_ORIGIN=true`, which rejects WebSocket upgrades whose `Origin` header is not on the allowlist. The operator UI is served over HTTPS by Caddy with a locally generated certificate authority; the operator browser is expected to trust that CA during field operation.

11.2 gRPC Surface

gRPC listens only on the loopback interface. Cross-host traffic to the UI terminates at Caddy and is multiplexed over the WebSocket binary envelope; gRPC-web calls from the UI are proxied to loopback gRPC by the gateway.

11.3 Firmware Update Integrity

Firmware images are CRC-32-checked before and after transfer to the RX72N. A signature field is reserved in `firmware_update.proto` for future cryptographic signing but is not yet enforced (see Section 14).

11.4 Service Sandboxing

The cockpit API and gateway systemd units use `NoNewPrivileges=true`, `PrivateTmp=true`, `ProtectSystem=stri` and a restricted `ReadWritePaths` set so the services cannot modify anything outside their state directory.

11.5 Character Encoding as Defense

The 7-bit ASCII enforcement hook prevents the introduction of confusable Unicode characters, homograph attacks in identifiers, and smuggled payloads in string literals.

12 Performance Considerations

12.1 Real-Time Budgets

- Motor control loop: 250 Hz servo (4 ms period); PID math completes in approximately 2 µs at 240 MHz with `-O2`.
- Watchdog monitor: feeds WDT at 100 Hz (10 ms period).
- SPI link budget: 10 MHz raw, 10 Mbit/s wire throughput; after rate-1/2 FEC and framing the useful payload is approximately 4.5 Mbit/s sustained.
- Hardware CRC-32: approximately 32 µs per 100 bytes on the RX72N hardware CRC peripheral.
- ROS2 EKF: 50 Hz cycle.
- UI rendering: 60 Hz via uPlot with GPU-accelerated canvas draws.

12.2 Known Bottlenecks

LiDAR USB jitter can spike scan latency to tens of milliseconds under contention; the SLAM async pipeline absorbs this. ROS2 DDS discovery storms can transiently saturate loopback on bringup; this is contained by a staggered launch sequence in `star_bringup`.

12.3 Scalability Limits

The current firmware supports exactly four motors, sized to the available GPTW channels and dead-time generators. SRAM usage is bounded statically and the margin at link time is less than 5 % of the 512 kB total; adding a fifth motor would require pruning the debug logging buffer or upgrading to a larger-SRAM RX variant.

13 Testing Strategy

13.1 Unit Tests

- RX72N firmware: Unity v2.6.1 auto-fetched by CMake; approximately 35 test executables. Mocks cover `bus_interface`, `motor`, `uart`, `error_handler`, and ThreadX primitives via `MOCK_TX_THREAD_CREA`
- Gateway: Go's built-in `testing` plus `testify` for nanopb bounds, frame codec, FEC, and Viterbi.
- UI: Vitest plus `@testing-library/react` for components and Zustand slice reducers.
- Compliance engine: `pytest` for the engines and golden outputs for the report generator.

13.2 Integration Tests

A `virtual_rx72n` simulator reimplements the RX72N SPI peripheral surface in-process so the gateway can execute end-to-end protocol tests without hardware. ROS2 launch tests exercise startup orchestration; gRPC conformance tests exercise the service surface.

13.3 Hardware Bring-Up

Dedicated bring-up executables live in the firmware tree: `motor_spin_test/`, `pwm_test/`, `encoder_test/`, `gpio_test/`, `uart_test/`, `imu_test/`, and `sonar_test/`. These are built by the firmware CMake presets and flashed individually.

13.4 CI/CD

GitHub Actions orchestrates all builds: firmware, gateway, UI, ROS2 (via Docker), and MATLAB linting. Pre-commit hooks enforce 7-bit ASCII and basic formatting; CodeRabbit performs AI code review on every pull request.

14 Known Limitations and Future Work

- **Four-motor ceiling.** Firmware is sized to exactly four GPTW PWM channels and four TPU encoder channels. A fifth motor requires hardware expansion.
- **Single-robot SLAM.** `slam_toolbox` supports only a single robot in the current configuration. Multi-robot deployments would require cooperative SLAM stacks such as Kimera-Multi or Swarm-SLAM.
- **Global planner A* only.** Nav2 is configured with its default A* grid planner. More sophisticated planners such as Hybrid A* or state-lattice planners are future work for non-rectilinear environments.
- **Firmware update signing.** The firmware-update path verifies CRC-32 only. Reserving a signature field in the proto is a first step; full signed-update verification with a public-key store on the RX72N is future work.
- **Single-operator UI.** The dashboard assumes one operator; multi-operator arbitration, role separation, and audit logging are not implemented.
- **PID gain design requires an offline MATLAB re-run for any plant model change; no online autotuning.** If the motor or the load changes significantly, the MATLAB workflow must be re-executed and the firmware rebuilt. STAR does not perform online PID autotuning.
- **English-only UI.** Internationalization and localization infrastructure (i18n bundles, RTL layout) are not present.
- **Compliance coverage.** Only the ramp-slope check is fully implemented; path width and trip hazard are stubs and the remaining ADA checks are specified but not coded.

Appendices

Appendix A: Glossary

RX72N
: A Renesas 32-bit microcontroller with a 240 MHz RX CPU core, 4 MB of flash, 512 KB of SRAM, and an extensive on-chip peripheral set used for STAR motor control.

ThreadX
: A real-time operating system from Microsoft (formerly Express Logic) distributed as part of Azure RTOS, used on the RX72N.

HARQ
: Hybrid Automatic Repeat Request, a combination of forward error correction and automatic retransmission; Chase Combining HARQ accumulates soft bits across retransmissions.

FEC
: Forward Error Correction. STAR uses a rate-1/2 convolutional code with constraint length

7 and generators `171/133` octal, decoded by soft-decision Viterbi.

EKF Extended Kalman Filter, a first-order linearization of the Kalman filter used in `robot_localization` for odometry/IMU fusion.

DDS Data Distribution Service, the publish/subscribe middleware that underpins ROS2 transport.

RANSAC
Random Sample Consensus, an iterative robust estimator used by the compliance engine for plane fitting over LiDAR point clouds.

REP-105
ROS Enhancement Proposal 105, which prescribes the canonical TF frame hierarchy `map -> odom -> base_link`.

Appendix B: References

1. U.S. Department of Justice, *2010 ADA Standards for Accessible Design*, Section 405.2 (Ramp Running Slope), 15 September 2010. https://www.access-board.gov/ada/chapter/ch04/
2. Seyfarth Shaw LLP, "ADA Title III Federal Lawsuit Numbers Rebound to 8,800 in 2024," ADA Title III blog, March 2025. https://www.adatitleiii.com/2025/03/ada-title-iii-federal-lawsuit-nu
3. G. J. Holzmann, "The Power of 10: Rules for Developing Safety-Critical Code," *IEEE Computer*, vol. 39, no. 6, pp. 95–99, June 2006. https://spinroot.com/gerard/pdf/P10.pdf
4. Boston Dynamics, "API Protobuf Guidelines," Spot SDK. https://github.com/boston-dynamics/spot-sdk/blob/master/docs/protos/style_guide.md
5. Open Navigation LLC, Nav2 documentation. https://docs.nav2.org/
6. nanopb, Petteri Aimonen et al. https://github.com/nanopb/nanopb
7. Eclipse ThreadX, Eclipse Foundation (donated by Microsoft, 2024). https://threadx.io/
8. Open Robotics, ROS 2 Jazzy Jalisco release (LTS, May 2024 – May 2029, Ubuntu 24.04). https://docs.ros.org/en/jazzy/
9. S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, art. 2783, 2021. https://doi.org/10.21105/joss.02783

Appendix C: Traceability Matrix (ADA Checks)

Table 8 reproduces the machine-maintained traceability matrix in `final_capstone/extras/traceability.md`. Each row maps one ADA check to the sensor feeding it, the algorithm used, the implementing ROS2 node, the published topic, and the status label (using the legend defined in Section 5.8).

Table 8: Compliance-engine traceability matrix.

ADA check	Sensor	Algorithm	Node	Status
405.2 Ramp slope	LiDAR + IMU	RANSAC plane + IMU cross-check	ramp_slope_node	[IMPLEMENTED]
303 Trip hazard	LiDAR	Vertical discontinuity detector	trip_hazard_node	[STRETCH]
403.5 Path width	LiDAR map	Cross-section width along path	path_width_node	[STRETCH]
405.5 Ramp width	LiDAR	Lateral extent of ramp plane	ramp_width_node	[ARCHITECTED]
405.7 Ramp landings	LiDAR	Adjacent plane detection	ramp_landing_node	[ARCHITECTED]
404.2.3 Door clear width	LiDAR	Doorway passable-width scan	door_width_node	[ARCHITECTED]
404.2.5 Door threshold	LiDAR	Threshold height measurement	door_threshold_node	[ARCHITECTED]

Appendix D: Compliance Checklist

☐ All source files pass the 7-bit ASCII pre-commit hook.

☐ Firmware builds with `-Wall -Wextra -Werror` without warnings.

☐ Gateway `go vet` and `golangci-lint` pass.

☐ ROS2 workspace builds cleanly under `colcon build -symlink-install`.

☐ UI builds with `vite build` and passes `tsc -noEmit`.

☐ All `.proto` files pass `buf lint` and `buf breaking` against main.

☐ Compliance engine `pytest` suite passes.

☐ MATLAB PID design script reproduces the current `pid_config_output.txt` bit-for-bit.

☐ Doxygen generation produces zero warnings.